MACHINE LEARNING · CLASSIFICATION

The Naïve Bayes Classifier

Reasoning under uncertainty — fast, simple, and surprisingly hard to beat

Course: Machine Learning (CS-XXX)

Program: BS CS / SE / AI — 5th Semester

Session: Fall 2026

Sukkur IBA University · Department of Computer Science

A STORY BEFORE THE FORMULA

It's Saturday morning. The sky is overcast. Do you go play badminton?

You've played on days like this before — sometimes it worked out, sometimes it didn't. You don't have certainty. What you have is **evidence** and **experience**. That is exactly the problem Naïve Bayes solves.

By the end of this lecture you will be able to answer, with actual numbers:

1. Given it's overcast, what's the probability you'll play?
2. How does adding “mild temperature” change that answer?
3. Why does one zero in your data almost break the model — and how do we fix it?

Play Badminton Dataset

14 historical Saturdays (Mitchell, Machine Learning). Every row: the weather that day, and whether the game happened.

Weather	Play?
Sunny	No
Sunny	No
Overcast	Yes
Rainy	Yes
Rainy	Yes
Rainy	No
Overcast	Yes
Sunny	No
Sunny	Yes
Rainy	Yes
Sunny	Yes
Overcast	Yes
Overcast	Yes
Rainy	No

What we'll build, in order

01 **Classification, recapped**

Binary vs. multiclass, and predict vs. predict_proba

02 **Why exact Bayesian reasoning breaks down**

The computational wall of joint probabilities

03 **The naive assumption**

How one simplifying assumption rescues the math

04 **Worked examples, fully corrected**

One feature, then several — with the right labels

05 **The zero-frequency problem**

What happens when your data has never seen a case

06 **The Naive Bayes family & when to use it**

Gaussian, Multinomial, Bernoulli — and real limitations

Classification: predicting a category, not a number

Binary classification — predict a 1 or a 0: phishing / not-phishing, churn / no churn, click / no click.

Multiclass classification — predict one of several classes, often as a probability per class, e.g. a tumor as normal, benign, or malignant.

Two outputs you'll use constantly:

`.predict(x)`	Returns the class label itself
`.predict_proba(x)`	Returns a probability for each class

The sliding cutoff

A probability becomes a decision only after you pick a cutoff.

Higher cutoff → fewer records labeled “1” (stricter)

Lower cutoff → more records labeled “1” (looser)

General approach: estimate P(class), compare to cutoff, assign the record above it.

Naïve Bayes: fast, simple, everywhere

A probabilistic classifier built on Bayes' theorem. It doesn't try to be the most accurate model in the room — it tries to be the fastest reasonable one, and on high-dimensional, sparse data it's often hard to beat.

Spam filtering

Is this email spam, given the words it contains?

Text classification

Sort documents into topics using word frequencies.

Sentiment analysis

Positive or negative review, from word evidence.

Recommender systems

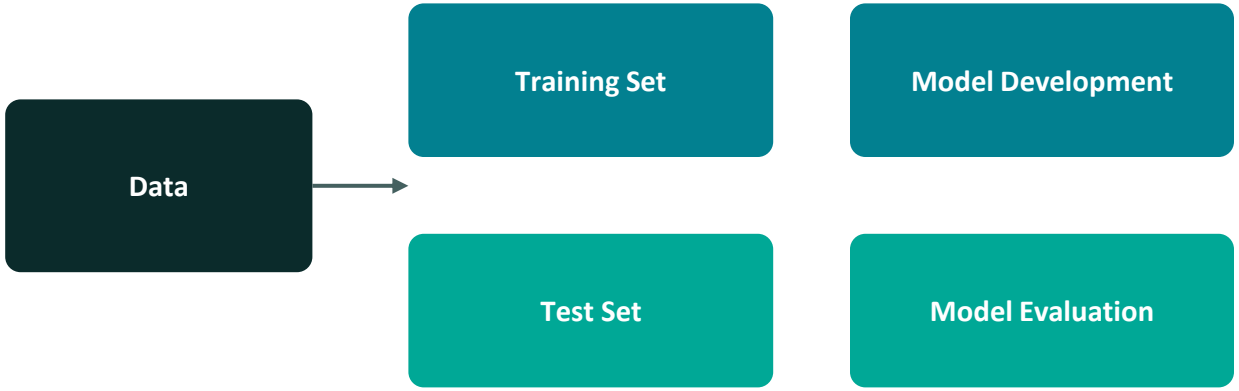
Estimate the chance a user likes an item.

Before you classify anything: identify your features (the attributes that might affect the outcome) and your label (the outcome itself). For a bank deciding on a loan, features might be occupation, income, age, location, and credit history — the features that help the model decide.

Learning phase, then evaluation phase

Learning phase — the classifier trains on a dataset, building its frequency and probability tables.

Evaluation phase — the trained model is tested on unseen data, and performance is measured with accuracy, precision, recall, and error rate.

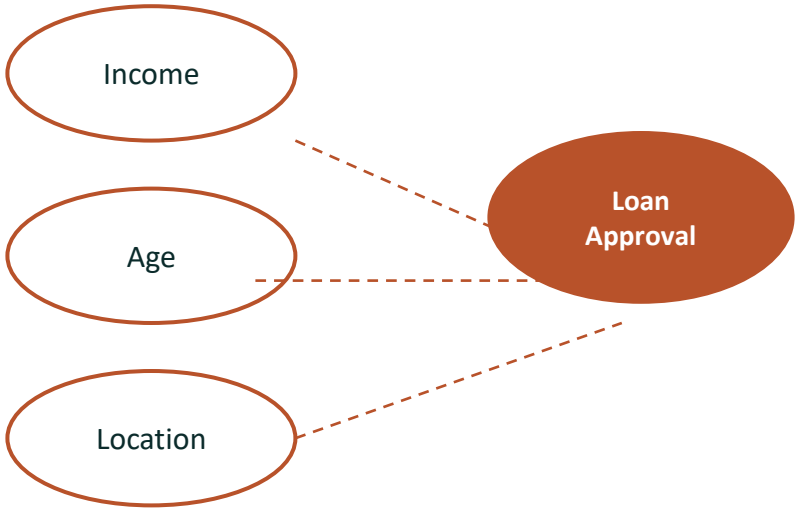


Performance measures

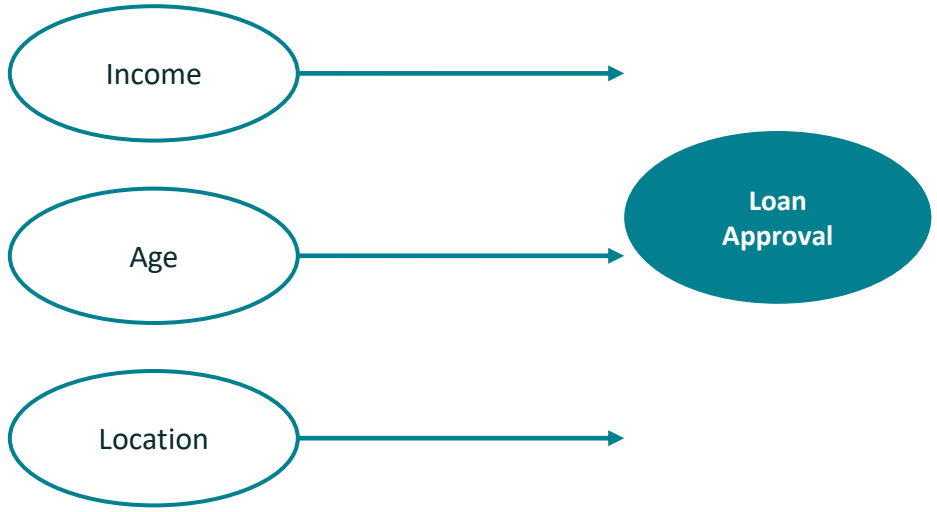
- Accuracy — overall fraction correct
- Precision — of predicted positives, how many were right
- Recall — of actual positives, how many were caught
- Error rate — the complement of accuracy

The 'naive' assumption solves a computational bottleneck

Reality: features are interdependent



The naive hack: class-conditional independence



Naïve Bayes assumes each feature contributes to the class **independently of every other feature** — even when, in reality, income and age and location are correlated. This is called **class-conditional independence**, and it's why the method is called “naive.” It turns an intractable joint-probability computation into a simple product of terms.

Bayes' theorem: updating belief with evidence

$$P(A \mid B) = P(B \mid A) \cdot P(A) \;/\; P(B)$$

$P(A \mid B)$

POSTERIOR

What we want: probability of hypothesis A, now that we've observed evidence B.

$P(B \mid A)$

LIKELIHOOD

Probability of seeing evidence B, assuming hypothesis A is true.

$P(A)$

PRIOR

Probability of hypothesis A before seeing any evidence.

$P(B)$

MARGINAL

Overall probability of the evidence, across all hypotheses.

Keep these four names straight — the single most common mistake in Naive Bayes worked examples (including in many popular tutorials) is calling a likelihood a "posterior."

METHOD

Naive Bayes classification, step by step

1

Prior

Calculate P(class) for every class label, from raw frequencies.

2

Likelihood

For each feature, calculate P(feature | class) — one likelihood per feature per class.

3

Posterior

Multiply likelihoods × prior (Bayes' rule) to get an (unnormalized) posterior score for each class.

4

Decide

Pick the class with the higher posterior score. That's the prediction.

Two bookkeeping tools make this easy: **the frequency table** (raw counts of each feature value per class) and **the likelihood table** (those counts converted to conditional probabilities P(feature | class)).

From raw counts to likelihoods (not posteriors)

Frequency table

Weather	No	Yes
Overcast	0	4
Sunny	2	3
Rainy	3	2
Total	**5**	**9**

Likelihood table — P(Weather | Play)

Weather	P(w|No)	P(w|Yes)
Overcast	0/5 = 0	4/9 = 0.44
Sunny	2/5 = 0.40	3/9 = 0.33
Rainy	3/5 = 0.60	2/9 = 0.22

Prior probabilities — P(Play)

Class	P
No	5/14 = 0.36
Yes	9/14 = 0.64

Will they play, given it's overcast?

Hypothesis: Yes

$$P(\text{Yes} \mid \text{Overcast}) = P(\text{Overcast} \mid \text{Yes}) \times P(\text{Yes}) \;/\; P(\text{Overcast})$$

Likelihood: $P(\text{Overcast} \mid \text{Yes}) = 4/9 = 0.44$
Prior: $P(\text{Yes}) = 9/14 = 0.64$
Marginal: $P(\text{Overcast}) = 4/14 = 0.29$

$P(\text{Yes} \mid \text{Overcast}) = 0.44 \times 0.64 / 0.29 =$ **0.98**

Hypothesis: No

$$P(\text{No} \mid \text{Overcast}) = P(\text{Overcast} \mid \text{No}) \times P(\text{No}) \;/\; P(\text{Overcast})$$

Likelihood: $P(\text{Overcast} \mid \text{No}) = 0/5 = 0$
Prior: $P(\text{No}) = 5/14 = 0.36$

$P(\text{No} \mid \text{Overcast}) = 0 \times 0.36 / 0.29 =$ **0**

Verdict

0.98 > 0

Given an overcast Saturday, the model predicts “Yes, play.” The evidence overwhelmingly favors it.

What if we have more than one feature?

Real problems rarely give you one clue. Suppose we know it's overcast AND mild. Bayes' theorem generalizes cleanly:

$$P(Y \mid X) = P(X \mid Y) \times P(Y) \;/\; P(X)$$

Now apply class-conditional independence — the naive assumption lets the joint likelihood factor into a simple product:

$$P(Y \mid X_1, X_2, \ldots, X_n) = [\, P(X_1 \mid Y) \cdot P(X_2 \mid Y) \cdot \ldots \cdot P(X_n \mid Y) \cdot P(Y) \,] \;/\; [\, P(X_1) \cdot P(X_2) \cdot \ldots \cdot P(X_n) \,]$$

For our example: $P(\text{Play=Yes} \mid \text{Overcast, Mild}) = P(\text{Overcast} \mid \text{Yes}) \times P(\text{Mild} \mid \text{Yes}) \times P(\text{Yes}) \;/\; P(\text{Overcast}) \times P(\text{Mild})$

Each additional feature just adds one more likelihood term to the product — that's the entire scaling trick.

Overcast AND Mild — the right way to normalize

Temperature likelihood table

Temp	No	Yes	P(No)	P(Yes)
Hot	2	2	2/5=0.40	2/9=0.22
Mild	2	4	2/5=0.40	4/9=0.44
Cool	1	3	1/5=0.20	3/9=0.33

Numerator for Yes = P(Overcast|Yes) × P(Mild|Yes) × P(Yes)

= 0.44 × 0.44 × 0.64 = 0.1240

Numerator for No = P(Overcast|No) × P(Mild|No) × P(No)

= 0 × 0.40 × 0.36 = 0.0000

Wait — a probability of exactly 1.00 should worry you. It happened only because P(Overcast | No) = 0 in our tiny 14-row dataset — nobody in the data skipped playing on an overcast day, purely by chance of a small sample. That single zero doesn't just lower the “No” score, it **annihilates it completely**, no matter what the other evidence says. This is the zero-frequency problem, and it's exactly what the next slide fixes.

The zero-frequency problem — and Laplace smoothing

The problem

If a feature value never appears with a class in training data, its likelihood is exactly zero.

Because Naive Bayes multiplies likelihoods together, a single zero forces the whole product to zero — **regardless of how strong the other evidence is.**

In our dataset, P(Overcast | No) = 0/5. Any future "Overcast" day is now mathematically impossible to classify as "No," forever — an artifact of a small sample, not a real law of weather.

The fix: Laplace (additive) smoothing

$$P(x \mid c) = (\text{count}(x,c) + \alpha) \,/\, (\text{count}(c) + \alpha \cdot k)$$

α — smoothing strength (α = 1 is the classic "add-one" version)
k — number of possible values that feature can take

Recomputed: $P(\text{Overcast} \mid \text{No}) = (0 + 1) / (5 + 1\times3) = 1/8 = 0.125$ — small, but no longer zero. Strong contrary evidence can now, correctly, still tip the decision toward "Yes."

scikit-learn detail: every NB class (GaussianNB, MultinomialNB, BernoulliNB) applies smoothing by default via the var_smoothing / alpha parameter — you rarely have to compute this by hand.

The Naive Bayes family

scikit-learn ships three flavors, distinguished by what kind of data they assume for P(feature | class).

GaussianNB

CONTINUOUS DATA

Assumes each feature follows a normal (Gaussian) distribution within each class.

e.g. sensor readings, exam scores, physical measurements

MultinomialNB

DISCRETE COUNTS

Assumes data is multinomially distributed — built for frequency counts.

e.g. word counts in a document; the industry standard for text classification

BernoulliNB

BINARY FEATURES

Assumes each feature is an independent yes/no indicator.

e.g. “does this word appear at all?” in spam detection

Where Naive Bayes wins, and where it doesn't

Strengths

- ✓ Extremely fast to train and predict — one pass over the data
- ✓ Works well with high-dimensional data (thousands of words)
- ✓ Needs relatively little training data to estimate parameters
- ✓ A strong, simple baseline — hard to beat on text classification
- ✓ Handles multiclass problems naturally

Limitations

- ✗ The independence assumption is almost always false in practice
- ✗ Zero-frequency features can silently break predictions without smoothing
- ✗ Poor probability calibration — predicted probabilities are often extreme (near 0 or 1), even when class ranking is correct
- ✗ Continuous features must fit an assumed distribution (e.g. Gaussian) or be discretized first

Naive Bayes in five lines

```
# load a dataset and split it
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=1)

# train Gaussian Naive Bayes
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# predict and evaluate
from sklearn.metrics import accuracy_score, confusion_matrix
y_pred = gnb.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
print(confusion_matrix(y_test, y_pred))
```

Output

```
Accuracy: 95.0%

Confusion matrix:
[[19  0  0]
 [ 0 19  2]
 [ 0  1 19]]
```

Same three ideas as our by-hand example: priors from class counts, likelihoods from feature distributions, multiply and compare. sklearn just does it for 4 features and 150 rows instead of 1 feature and 14.

So — do you play?

With P(Yes | Overcast) = 0.98 against P(No | Overcast) = 0, the model says yes, decisively. Add “mild” and, once we normalize correctly (and once we account for the zero-frequency problem with smoothing), the evidence still points the same way — just not with the false certainty of exactly 1.00.

1. Bayes' theorem turns prior belief + evidence into an updated (posterior) belief
2. Naive Bayes assumes features are class-conditionally independent — an approximation that makes the math tractable
3. Never confuse a likelihood P(feature|class) with a posterior P(class|feature)
4. Normalize by summing class numerators, not by multiplying raw feature marginals
5. Zero counts need Laplace smoothing, or one missing case can override all other evidence